Integers, Floating-Point Numbers and Arithmetic Calculations

Integers and floats are the primary numeric data types in Python. With them, you can store numeric data and perform mathematical operations.

Integers are whole numbers without decimal points, either positive or negative.

Floats are positive or negative numbers with decimal points, like `3.14`, `-0.5`, or `0.0`.

If you mix integers and floats while performing basic arithmetic operations, Python will return a float as the result.

The modulo operator (`%`) returns the remainder when the value on the left is divided by the value on the right.

Floor division divides two numbers and returns the greatest integer less than or equal to the result. This is done with the double forward slash operator (`//`).

Exponentiation raises a number to the power of another, and is done with the double asterisk operator (`**`).

```
a = 15
b = 5
c = 12
d = 23.5

addition = a + b + c # 32
subtraction = a - b # 10
multiplication = a * b # 75
division = a / c # 1.25
remainder = c % b # 2
floor_div = c // b # 2
exponent = b ** 3 #125
```

Type Casting

Type casting allows you to change the data type. It is done by wrapping the literal or variable with the whichever data type you want to cast it into.
`float(), int(), str().`

```python
str_int = '45'
str_float = '7.8'
my_float = 12.92563

str_converted_int = int(str_int) # 45
str_converted_float = float(str_float) # 7.8
float_to_int = int(my_float) # 12
```

Python methods for working with integer and floats

`round()`: Rounds a number to the specified number of decimal places. By default this function rounds to the nearest integer, and returns a whole number with no decimal places.

`abs()`: returns the absolute value of a number

`pow()`: raises a number to the power of another or performs modular exponentiation

```python
rounded = round(my_float, 2) # 12.93
absolute = abs(num_1) # 45
power = pow(2, 4) # 16
power_mod = pow(2, 4, 7) # 2
```

Augmented Assignments

Augmented assignment combines a binary operation with an assignment in one step. It takes a variable, applies an operation to it with another value, and stores the result back into the same variable.

```python
my_var = 10
my_var += 5

print(my_var) # 15
```

Every operator can use an augmented assignment. The same works for subtraction, multiplication, division, floor division, modulo, and exponentiation.

The multiplication assignment operator can be used to repeat a string.

```python
test = "Hello World "
print(test*3) # Hello World Hello World Hello World
```

Logical Operators and Conditional Statements

Comparison operators are operators that let you compare two or more values, and return a boolean value. These operators can be used in conditionals to compare values and run certain code based on whether the conditional evaluates to `True` or `False`.

Operator	Name	Description
`==`	Equal	Checks if two values are equal
`!=`	Not equal	Checks if two values are not equal
`>`	Greater than	Checks if the value on the left is greater than the value on the right
`<`	Less than	Checks if the value on the left is less than the value on the right
`>=`	Greater than or equal	Checks if the value on the left is greater than or equal to the value on the right
`<=`	Less than or equal	Checks if the value on the left is less than or equal to the value on the right

```
print(1 > 4) # False
print(1 < 4) # True
print(1 == 4) # False
print(1 == 4) # True
print(1 != 4) # True
print(1 >= 4) # False
print(1 <= 4) # True
```

Conditional statements, or conditionals, let you control the flow of your program based on whether certain conditions are true or false.

In Python, the conditional is the `if` statement. Here's the basic syntax:

```
if grade > 3.5:
    print("Exceptional!!")
```

- `if` statements start with the `if` keyword.
- `condition` is an expression that evaluates to `True` or `False`, followed by a colon (`:`).

- The body of the `if` statement constitutes a *code block*, which is a group of statements that belong together. In Python, the level of indentation is what defines a code block.

The `else` clause runs when the `if` condition is false. Here's the syntax for an `if…else` statement:

```
grade = 3.75

if grade > 3.5:
    print("Exceptional!!")
else:
    print("Try harder.")
```

Note that you cannot place any statements between the `if` block and the `else` clause. If you do, it will raise a `SyntaxError.`

To account for multiple conditions, you can extend your if statement with the `elif` (else if) keyword.

```
grade = 3.75

if grade > 3.5:
    print("Exceptional!!")
elif grade > 3.0:
    print("Good.")
else:
    print("Try harder.")
```

Truthy and Falsy Values

In Python, every value has an inherent boolean value, or a built-in sense of whether it should be treated as `True` or `False` in a logical context. Many values are considered **truthy**, that is, they evaluate to `True` in a logical context. Others are **falsy**, meaning they evaluate to `False`.

Here are a few falsy values:

- `None`

- `False`
- Integer `0`
- Float `0.0`
- Empty strings `""`

Other values like non-zero numbers, and non-empty strings are truthy.

If you want to check whether a value is truthy or falsy, you can use the built-in `bool()` function. It explicitly converts a value to its boolean equivalent and returns `True` for truthy values and `False` for falsy values. Here are a few examples:

```
print(bool(False)) # False
print(bool(0))  # False
print(bool('')) # False

print(bool(True)) # True
print(bool(1)) # True
print(bool('Hello')) # True
```

Boolean Operators

Boolean operators, which are also known as logical operators or Boolean operators. These are special operators that allow you to combine multiple expressions to create more complex decision-making logic in your code.

There are three Boolean operators in Python: `and`, `or`, and `not`.

The `and` operator takes two operands and returns the first operand if it is falsy, otherwise, it returns the second operand. Both operands must be truthy for an expression to result in a truthy value.

The `and` operator takes two operands and returns the first operand if it is falsy, otherwise, it returns the second operand. Both operands must be truthy for an expression to result in a truthy value.

The `or` operator returns the first operand if it is truthy, otherwise, it returns the second operand. An `or` expression results in a truthy value if at least one operand is truthy.

This operator returns the first operand if it is truthy, otherwise, it returns the second operand. An `or` expression results in a truthy value if at least one operand is truthy.

The last operator we will look at is the `not` operator which takes a single operand and inverts its boolean value. It converts truthy values to `False` and falsy values to `True`. Unlike the previous operators we looked at, `not` always returns `True` or `False`.

```
is_weekend = True
is_sunny = False
temperature = 25  # in Celsius
has_umbrella = True


# 'and' example: Both must be true
if is_weekend and temperature > 20:
    print("It's a warm weekend, let's go out!")

# 'or' example: Only one needs to be true
if is_sunny or has_umbrella:
    print("You are prepared for the weather.")

# 'not' example: Flips the boolean value
if not is_sunny:
    print("It is currently cloudy.")


# Combining operators to handle more complex scenarios
# This checks if it's a nice day OR if it's the weekend,
# provided it isn't raining (not sunny)
if (is_sunny or is_weekend) and temperature >= 20:
    print("Conditions are perfect for a park visit.")

# Checking for a specific 'stay inside' condition
if not is_sunny and not is_weekend:
    print("It's a gray workday. Better stay focused inside.")
```